Structural-Based Testing Assignment

Jawaher Alzahrani

ID:1238299428

CSE 565: Software Verification and Validation

04/10/2026

Introduction

In this project we used testing and analysis techniques differently. There were test cases created to check the program behavior and get coverage information. Also, static analysis tool was applied to find problems in code without running the program.

Part 1:

Description of the Tool and Type of Coverage

In this assignment, JaCoCo was used within IntelliJ IDEA to assess code coverage. It is an open-source tool for Java programs that checks which parts of the code are run during testing and produces coverage results based on that. The results are shown in the IDE, where green indicates code that has been run during testing, while red indicates uncovered parts.

The following types of coverage are provided by the tool:

Statement coverage: determines what percentage of code lines were executed during testing.

Branch coverage: shows whether both outcomes of each decision (such as if/else conditions) were executed.

Method coverage: identifies which methods were invoked during testing.

Class coverage: indicates which classes were utilized during execution.

Condition coverage: checks if each condition in a decision has been evaluated as both true and false

Instruction coverage: shows the percentage of instructions that were executed during testing.

These metrics show how effective the test cases are and highlight any missing cases

Development and description of a set of test cases

A comprehensive set of test cases was developed to evaluate the behavior of the VendingMachine system under different input conditions. The test cases were designed based on the program logic and the possible range of user inputs. Specifically, test cases were implemented for each product (candy, coke, and coffee) to cover three main scenarios: exact amount, excess amount, and insufficient amount. For each product, a test case was created to verify correct dispensing when the exact price is entered, another to check that change is returned when more money is inserted, and a third to ensure that the item is not dispensed when the amount is insufficient. In addition, an edge case test was added where zero money is entered to examine how the system behaves under this condition. These test cases were designed to cover the main execution paths of the program and to validate the system's behavior under typical input conditions, contributing to achieving high statement and decision coverage. Figures 1–3 show the implemented test cases.

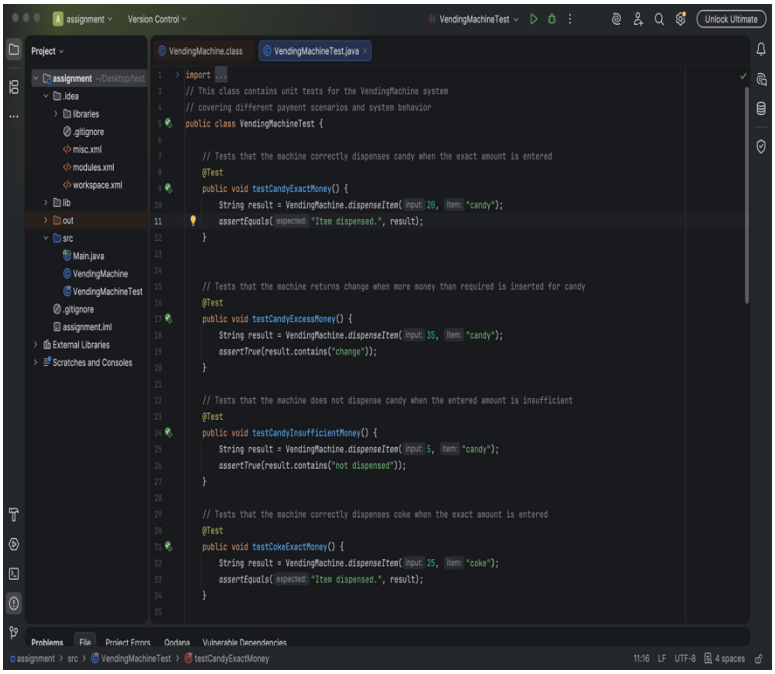
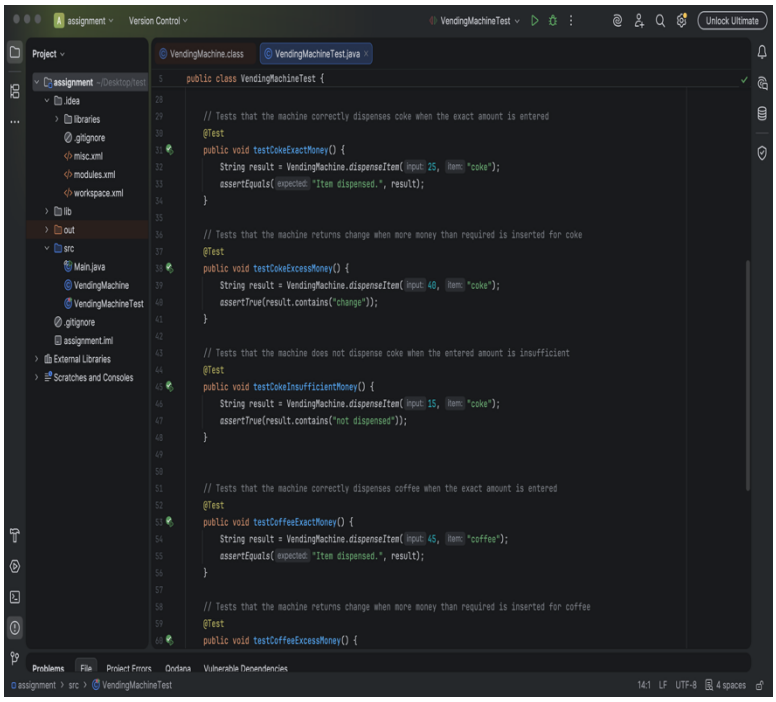
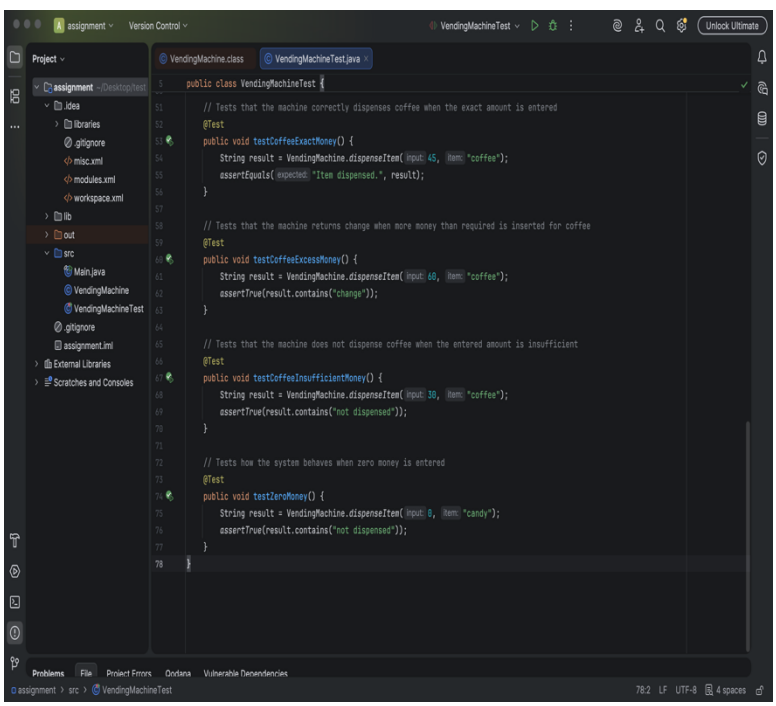


Figure 1 Test cases for different products (candy, coke, and coffee)



```java
public class VendingMachineTest {

    // Tests that the machine correctly dispenses coke when the exact amount is entered
    @Test
    public void testCokeExactMoney() {
        String result = VendingMachine.dispenseItem( input: 25, item: "coke");
        assertEquals( expected: "Item dispensed.", result);
    }

    // Tests that the machine returns change when more money than required is inserted for coke
    @Test
    public void testCokeExcessMoney() {
        String result = VendingMachine.dispenseItem( input: 40, item: "coke");
        assertTrue(result.contains("change"));
    }

    // Tests that the machine does not dispense coke when the entered amount is insufficient
    @Test
    public void testCokeInsufficientMoney() {
        String result = VendingMachine.dispenseItem( input: 15, item: "coke");
        assertTrue(result.contains("not dispensed"));
    }


    // Tests that the machine correctly dispenses coffee when the exact amount is entered
    @Test
    public void testCoffeeExactMoney() {
        String result = VendingMachine.dispenseItem( input: 45, item: "coffee");
        assertEquals( expected: "Item dispensed.", result);
    }

    // Tests that the machine returns change when more money than required is inserted for coffee
    @Test
    public void testCoffeeExcessMoney() {
```

Figure 2 Test cases for different products (continued)



```java
public class VendingMachineTest {
    // Tests that the machine correctly dispenses coffee when the exact amount is entered
    @Test
    public void testCoffeeExactMoney() {
        String result = VendingMachine.dispenseItem( input: 45, item: "coffee");
        assertEquals( expected: "Item dispensed.", result);
    }

    // Tests that the machine returns change when more money than required is inserted for coffee
    @Test
    public void testCoffeeExcessMoney() {
        String result = VendingMachine.dispenseItem( input: 60, item: "coffee");
        assertTrue(result.contains("change"));
    }

    // Tests that the machine does not dispense coffee when the entered amount is insufficient
    @Test
    public void testCoffeeInsufficientMoney() {
        String result = VendingMachine.dispenseItem( input: 30, item: "coffee");
        assertTrue(result.contains("not dispensed"));
    }

    // Tests how the system behaves when zero money is entered
    @Test
    public void testZeroMoney() {
        String result = VendingMachine.dispenseItem( input: 0, item: "candy");
        assertTrue(result.contains("not dispensed"));
    }
}
```

Figure 3 Test cases for different products and edge case (zero input)

Reporting out and discussion of the code coverage

Running all test cases with JaCoCo code coverage tool allowed measuring how much of the VendingMachine program was actually tested. From the analysis results, statement coverage was found to be 100% and branch coverage to be 93%. 100% statement coverage indicates that all lines of code were executed at least once. This means that the test cases covered the entire program. Through testing every condition once, 93% of branches were exercised. Branches refer to different paths of program execution. Testing every product with exact, excess, and insufficient amount conditions has covered every possible path of execution for each product. Achieving 100% branch coverage was not possible because of one condition that was not tested. There was no test case for an invalid product. So, the branch where invalid product is selected is not covered. Hence test cases meet the requirement of the assignment by providing 100 % statement coverage and more than 90% branch coverage. Figures 4,5 show the executed test cases, while Figure 6 shows the coverage report

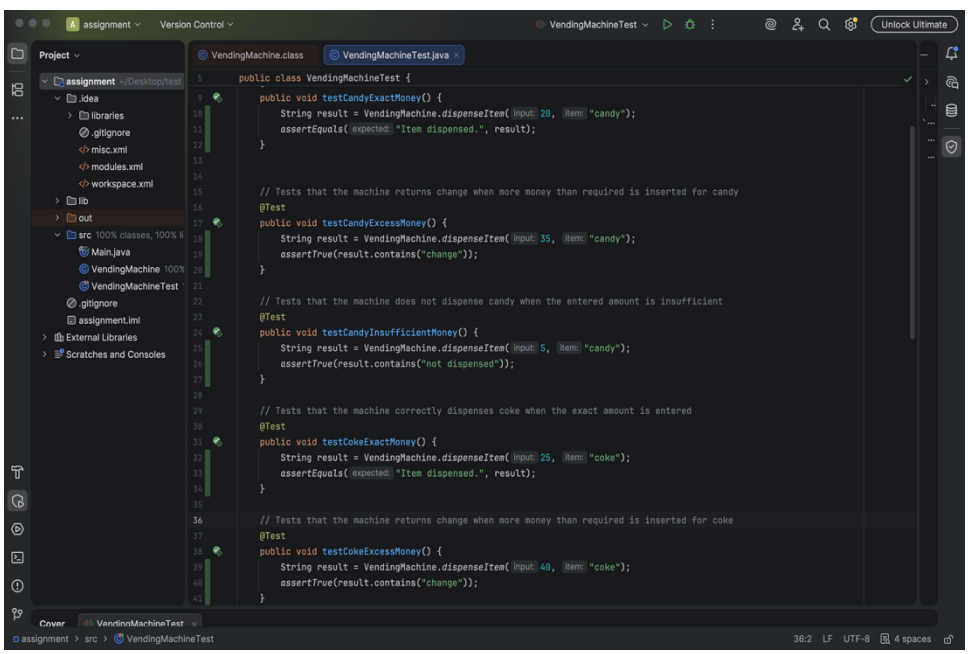


Figure 4 Execution of test cases showing successful results

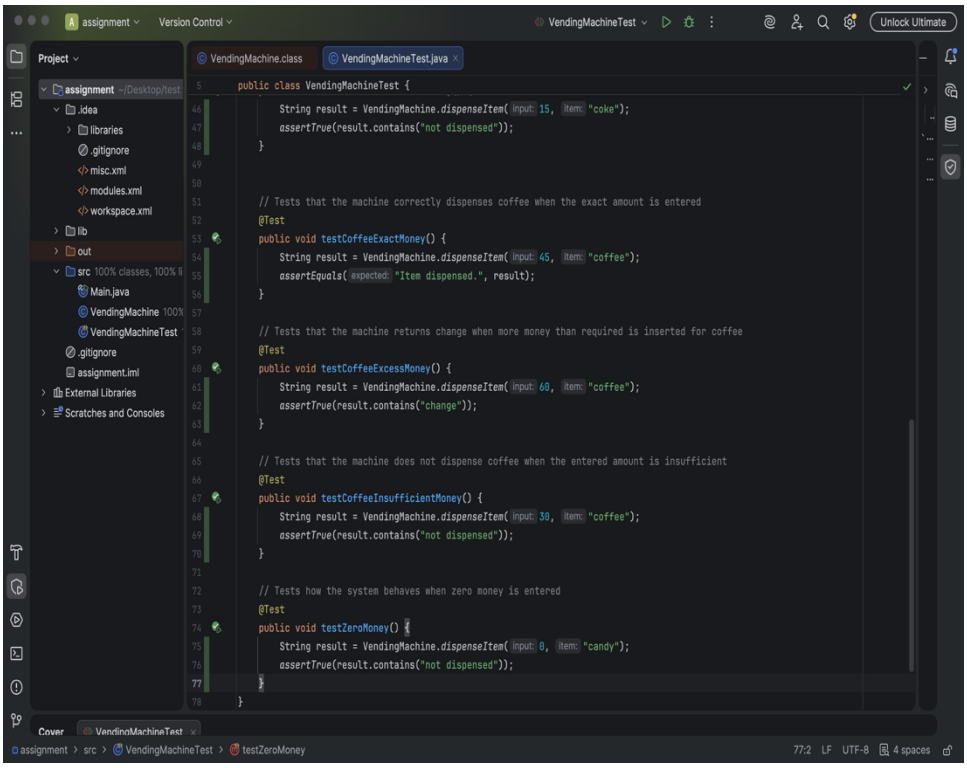


Figure 5 Execution of test cases showing successful results

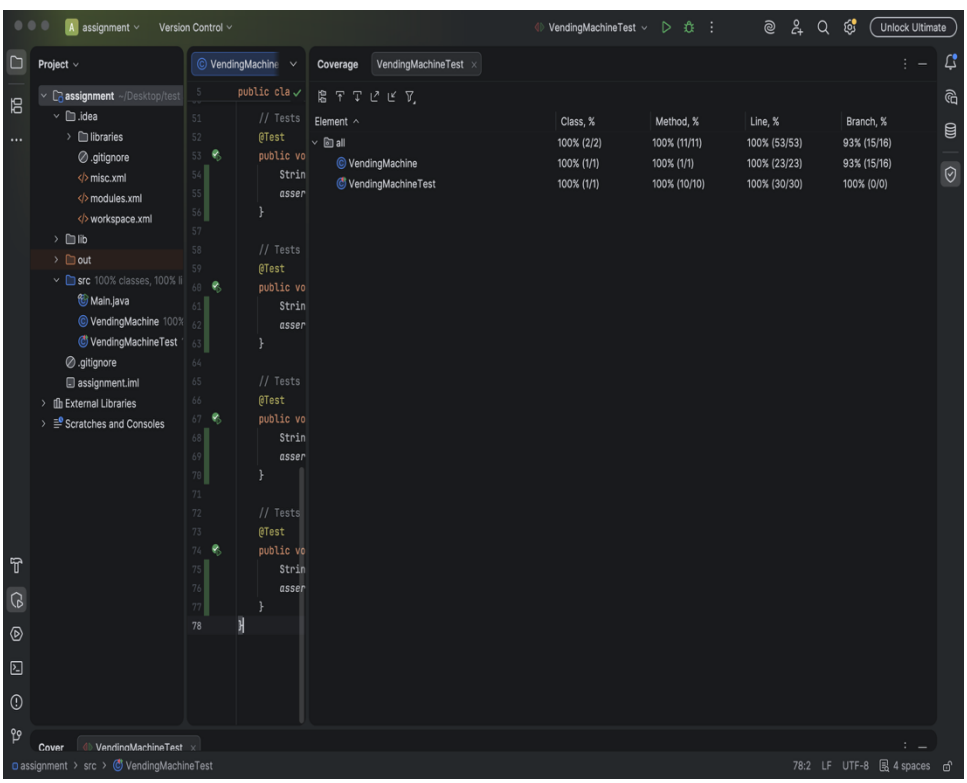


Figure 6 Code coverage results showing statement and branch coverage using JaCoCo

Part2

Description of the Tool and Types of Analysis

SonarQube for IDE is a static code analysis tool that allows one to examine code without executing it. The plugin is accessible within IntelliJ IDEA and checks your code for possible issues. when scanning StaticAnalysis.java, SonarQube detects different types of problems, including bugs, logic errors, and dataflow issues. It runs in real time as you code by highlighting errors in the editor. This allows you to easily identify problems in created code. It also produces a summary of the problems and detections, including unused variables, code smells, and logic errors.

The tool covers multiple key areas:

- Data flow issues (e.g., unused or improperly used variables)
- Code smells and maintainability issues
- Code style and formatting issues

Description and Analysis of Detected Anomalies

The SonarQube analysis identified several issues in the StaticAnalysis.java file, including:

- Unused variables such as weight and length
- An unused parameter (args)
- Redundant variable initialization (such as cost and output)
- Incorrect string comparison using == instead of equals()
- Use of System.out instead of a logger
- Some unnecessary operations, such as extra conversions

The analysis shows several issues that impact the quality of the code. Unused variables and parameters, such as weight and length (lines 15–16), reduce readability and represent definition without use data flow anomalies. Redundant initializations, such as cost (line 21), are forms of redefinition without use, where values are assigned and then overwritten before being used, leading to unnecessary operations. One of the most important issues is the incorrect use of the == operator for string comparison, which may result in incorrect behavior because it compares references instead of actual values. The tool also identified the use of System.out , which is not recommended compared to proper logging methods. In addition, some operations in the code can be simplified. Overall, these issues decrease code clarity, efficiency, and correctness, and addressing them would enhance the overall quality of the program. A comparison was done between IntelliJ Inspect Code and SonarQube using the same file. IntelliJ gives quick feedback in the editor and indicates issues like unused variables and incorrect comparisons during coding. In contrast, SonarQube provides a more detailed and structured analysis by classifying issues such as bugs and code smells, with clearer explanations. Overall, it provides a more complete evaluation of code quality. As shown in Figures 7-8, the analysis tools highlight various code issues.

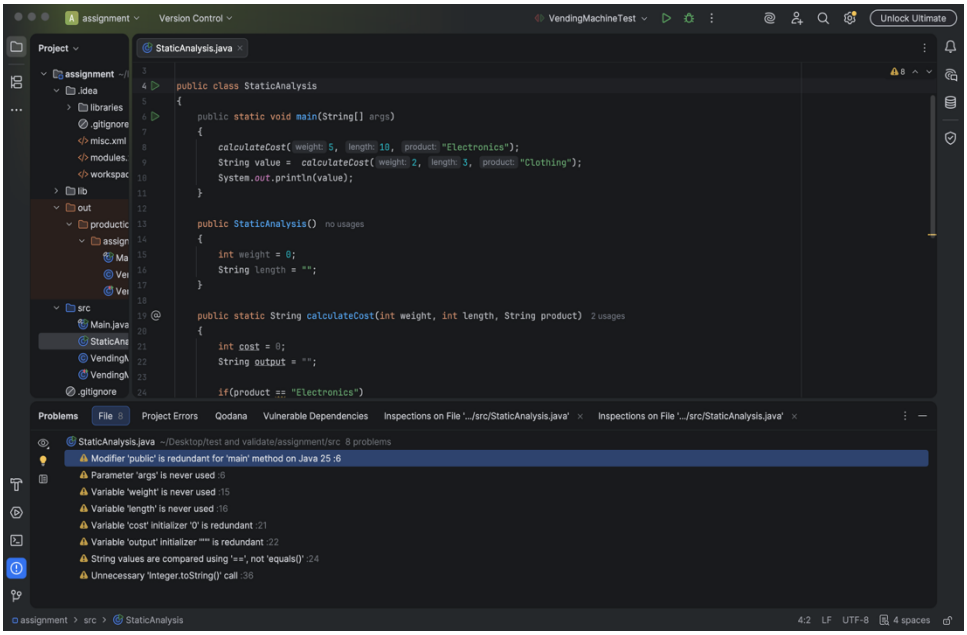


Figure 7 issues detected using IntelliJ built-in static analysis tool.

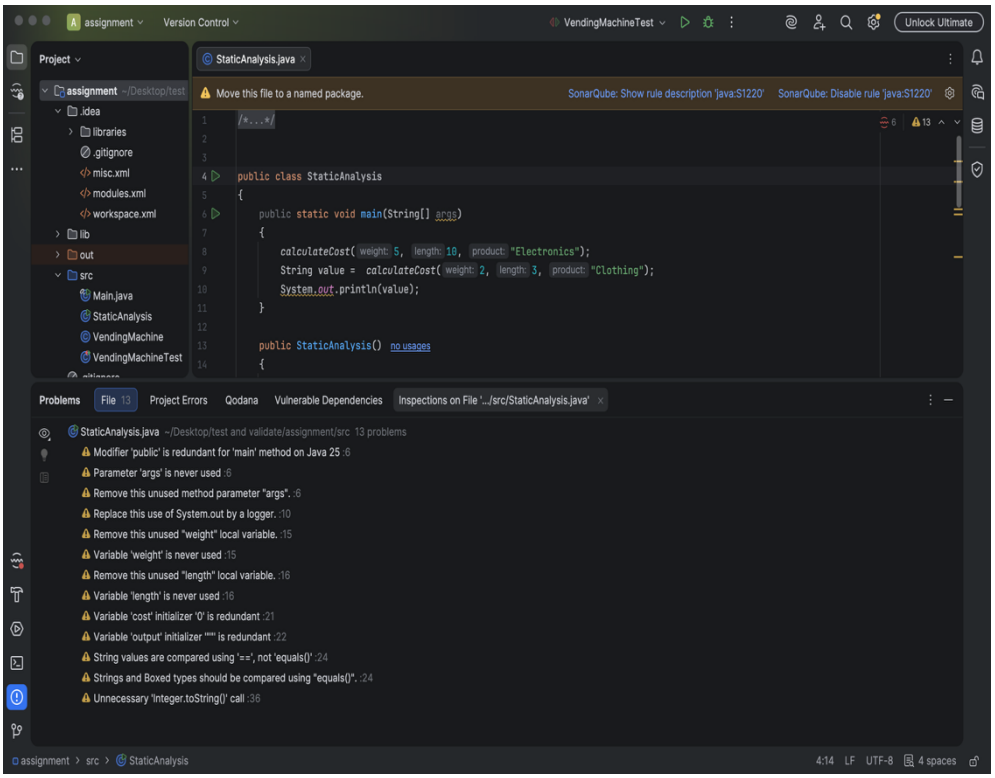


Figure 8 detected issues using SonarQube static analysis.

Assessment of the Tool

Features and Functionalities

SonarQube is useful for me because it analyzes code. It checks the code while it is being written. It runs inside the IDE. It displays problems in the editor: bugs, data flow problems, code smells, and stylistic errors. The most beneficial part of it explains the error in simple terms and provides possible solutions to the problems .This makes the issue clear and can help fix code one error at a time. Since it integrates with development tools like IntelliJ IDEA, it doesn't interrupt code writing.

Coverage Provided

SonarQube focuses on code quality, so it doesn't offer coverage details like statement or branch metrics, unlike testing tools. it identifies common coding problems like bugs, unused variables, and code smells, which helps developers to produce cleaner code and spot issues early .

Ease of Use

SonarQube is simple to set up, and beginners can use it. It automatically runs as a plugin through IntelliJ. Additionally, it presents problems clearly and provides easily interpretable messages so it's simple to correct issues. It's time-efficient and helps to write better and cleaner code with minimal efforts.